



SMART ATTENDENCE SYSTEM

USING RFID

Submitted by

BRISTI BHATTACHARJEE

Roll Number: 231001271196

SWARNABHA CHAKRABORTY

Roll Number: 231001271168

LABANI DAS

Roll Number: 231001271025

SAYAN RAJAK DAS

Roll Number: 231001271176

Supervisor

KISHALAY BAIRAGI

MASTER OF COMPUTER APPLICATION

2023-2025

CERTIFICATE

This is to certify that the dissertation/project entitled “Smart Attendance System” done by Bristi Bhattacharjee (231001271196), Swarnabha Chakraborty (231001271168), Labani Das (231001271025) and Sayan Rajak Das (231001271176) is a Bonafide work done under my supervision and guidance. This work has not been submitted elsewhere. This project is completed and submitted as a partial fulfilment of their post graduate course Master of Computer Application.

Supervisor Name : Kishalay Bairagi

Signature with Date

ACKNOWLEDGEMENT

Firstly, we would like to express our special thanks of gratitude to our teacher and mentor in this project Mr. Kishalay Bairagi for constantly supervising our paper correcting all faults and backlogs. With his support we were able to finish our paper within given time limit. Next, we would also like to extend our gratitude to our H.O.D. Anil Bikash Chowdhury for supporting and encouraging us for this project. Without such wonderful help and support we would have to face many difficulties.

Name: **Bristi Bhattacharjee**

Signature

Name: **Swarnabha Chakrobarty**

Signature

Name: **Labani Das**

Signature

Name: **Sayan Rajak Das**

Signature

Signature of Supervisor

Kishalay Bairagi

Signature of the Head of the Department

Anil Bikash Chowdhury

Table of Content

<u>Topic</u>	<u>Page number</u>
1. Abstract.....	5
2. Introduction.....	6-9
I. Domain description.....	6
II. Motivation.....	7
III. Scope of the work	8
3. Background / Review of related work	10-14
I. RFID card reader	10
II. NodeMCU	11
III. Buzzer	11
IV. Technical Specification of ESP8266.....	14
4. Methodology.....	15-16
I. Problem formulation.....	15
II. Algorithm description.....	16
5. Implementation.....	17-19
I. Hardware implementation.....	17
II. Software implementation.....	18
6. Result and Discussion.....	20-23
7. Future scope.....	24
8. Conclusion.....	25
9. References.....	26
10. Appendix.....	27-43
11. Fact sheet of components.....	44-49

ABSTRACT

This project aims to develop a smart attendance management system using **Node.js** and **RFID** (Radio Frequency Identification) technology. Traditional attendance systems often face challenges such as manual record-keeping, inaccuracies in tracking attendance, and inefficiencies in real-time data management. To address these issues, this project proposes an IoT-based solution that utilizes RFID tags for student identification and a web-based interface for streamlined attendance monitoring and reporting.

The system consists of two main components: the RFID-based hardware setup and the web application. RFID cards are assigned to students, and attendance is marked automatically when students scan their cards. The system stores the data in a centralized database, ensuring accurate and real-time record-keeping. Administrators and faculty can access detailed reports and track attendance patterns using the web-based interface built with **Node.js** and **EJS (Embedded JavaScript Templates)**.

Key features of the system include:

- Automatic card assignment with an unassigned card management system.
- Real-time attendance tracking and recording.
- An easy-to-use web interface for report generation and daily attendance records.
- Data visualization to assist in monitoring attendance trends and student participation.

By leveraging the combined power of **IoT** and modern web technologies, this project seeks to enhance the efficiency and accuracy of attendance systems. This innovative approach aims to reduce administrative workload and ensure a transparent, user-friendly attendance management process for educational institutions.

Introduction

Efficient attendance management is a critical aspect of modern educational institutions. Traditional systems, often reliant on manual entry, are prone to errors, time-consuming processes, and lack of real-time updates. These challenges result in inefficiencies and administrative burden, hindering effective attendance tracking and management.

This project aims to develop an intelligent attendance system that leverages **RFID (Radio Frequency Identification)** technology and modern web development tools such as **Node.js**. By integrating IoT principles, the proposed system seeks to provide an automated and seamless solution for managing attendance records. RFID tags assigned to students ensure accurate identification, while the system's real-time capabilities allow instant updates to centralized databases.

The objective of this project is to create a reliable, scalable, and user-friendly attendance management solution. It offers features such as card assignment, attendance tracking, and comprehensive reporting through an intuitive web interface. This innovative system is designed to minimize errors, save time, and enhance operational efficiency in educational institutions.

❖ Domain Description :

The domain of this project lies in **IoT-based attendance management systems** with applications in educational institutions. It combines technologies, principles, and methodologies from the fields of IoT, web development, and data management to create an efficient, automated system.

The primary focus of this domain is addressing the challenges associated with traditional attendance systems, including manual data entry, errors in record-keeping, and delays in processing attendance reports. The solution integrates **RFID** technology for student identification and **Node.js** for managing backend processes and web-based interfaces.

Key elements of the domain include:

1. **RFID Technology:** Utilized for seamless identification and tracking of students.
2. **Web-Based Interface:** Allows easy monitoring, reporting, and visualization of attendance records in real time.
3. **Database Management:** Centralized storage of attendance data ensures accuracy, security, and accessibility.
4. **Automation and Real-Time Processing:** Eliminates the need for manual entry, reducing errors and administrative workload.

The domain's applications are not limited to educational institutions. Similar systems

can be extended to workplaces, conferences, and other domains requiring attendance tracking. This project demonstrates how IoT and modern web development can be effectively utilized to replace traditional systems with intelligent and efficient alternatives.

❖ **Motivation :**

The development of a smart attendance management system is driven by several key motivations and benefits, addressing the challenges and inefficiencies of traditional attendance systems. The following are the primary motivations behind implementing such a system:

1. Automation of Attendance Process:

Traditional attendance systems often rely on manual record-keeping, which is time-consuming and prone to errors. By automating the attendance process with RFID technology, this project eliminates the need for manual entry, reducing inaccuracies and saving valuable time for both educators and students.

2. Real-Time Data Availability:

Manual systems lack the ability to provide real-time attendance updates. A smart attendance system offers instant tracking and recording of student attendance, ensuring that administrators and faculty have immediate access to attendance data when needed.

3. Reduction in Administrative Burden:

Managing attendance records manually requires significant administrative effort, including tallying records, tracking absences, and generating reports. By automating these tasks, this system minimizes the workload for faculty and staff, allowing them to focus on more impactful activities.

4. Improved Accuracy and Transparency:

Manual systems are prone to errors, such as duplicate entries or misplaced records. The use of RFID ensures accurate tracking of attendance data while maintaining transparency for all stakeholders, including students, parents, and administrators.

5. Enhanced Student Engagement:

A smart system encourages active participation by ensuring timely attendance marking and providing students with accountability for their own attendance. The integration of features like real-time notifications and personalized attendance reports adds value to the educational experience.

6. Data-Driven Insights for Decision-Making:

The system generates valuable data on attendance trends, student participation, and absenteeism patterns. This data can be analyzed to gain insights into student behavior,

enabling institutions to make informed decisions regarding interventions, resource allocation, and policy improvements.

7. Scalability and Versatility:

The smart attendance system can be easily scaled and adapted for various applications beyond education, such as conferences, workplaces, and public events. Its versatile design ensures applicability across diverse environments.

8. Sustainability and Eco-Friendly Practices:

Digital attendance management reduces the use of paper-based systems, contributing to environmental sustainability. By eliminating the need for physical attendance registers, this project supports eco-friendly practices while promoting efficient data management.

The motivation behind this project lies in creating an efficient, reliable, and user-friendly attendance management system that leverages technology to streamline operations, enhance accuracy, and reduce administrative overhead. By addressing the limitations of traditional systems, this project aims to revolutionize attendance tracking and provide a modern solution for educational institutions and beyond.

❖ Scope of the work :

The scope of work for the smart attendance management system includes a series of tasks and deliverables aimed at developing and implementing the project effectively. Key components of the scope are:

1. Requirement Analysis:

Conduct an in-depth analysis of user needs, including the operational environment, attendance tracking objectives, stakeholder expectations, and integration requirements.

2. System Design:

Create a detailed design encompassing hardware components (e.g., RFID readers, NodeMCU boards), software architecture, communication protocols, and data management workflows. Ensure scalability, reliability, and security.

3. Hardware Deployment:

Install and configure RFID readers and related devices for seamless attendance tracking. Ensure effective communication between hardware and the central server or cloud.

4. Data Collection and Processing:

Implement mechanisms to collect real-time attendance data and process it for accurate record-keeping. Develop analytics to derive insights on attendance trends and patterns.

5. User Interface Development:

Design and implement a user-friendly interface for administrators and users to access attendance records, track absences, and generate reports. Include features like notifications and attendance summaries.

6. Centralized Data Management:

Set up a central server or cloud platform to store, process, and manage attendance data. Ensure data privacy, security, and efficient access.

7. Integration and Testing:

Integrate all components—hardware, software, data processing, and user interfaces. Conduct comprehensive testing to validate the system's functionality, accuracy, and performance.

8. System Scalability and Adaptability:

Design the system to be scalable for use in various environments, such as educational institutions, workplaces, or conferences.

The defined scope ensures the smart attendance management system meets its objectives, offering a modern, efficient, and reliable solution for attendance tracking and management.

➤ BACKGROUND / REVIEW RELATED WORK

❖ RFID Card Reader:

A radio frequency identification system that uses tags or labels attached to the objects to be identified. Two-way radio transmitter receivers called interrogators or readers send a signal to the tag and read its response. There are two components of this system – RFID tags or labels and the RFID reader.



Power/Reset Pins

1. **Pin 1 (VCC):** This pin provides power to the module. In some versions of the RC522 module, this pin is labeled as 3V3 instead of VCC.
2. **Pin 2 (RST):** The reset pin is used to reset the device in case of errors or when the module is not responding.
3. **Pin 3 (Ground):** The ground pin establishes a common ground with external devices, such as the power supply, microcontroller, or Arduino.

SPI and UART Communication Pins

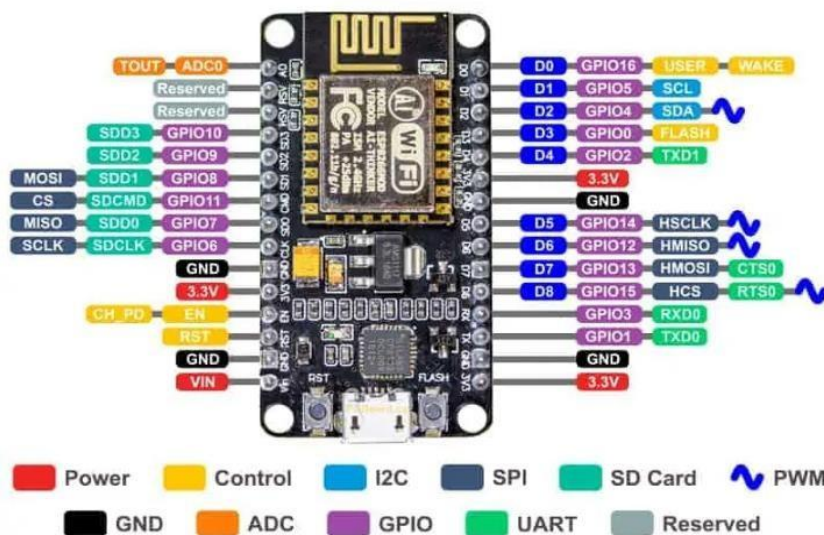
1. **Pin 4 (IRQ):** This pin is used to wake the device from sleep mode, helping to save power when the module is idle.
2. **Pin 5 (MISO/SCL/TX):** This pin is used in SPI communication to transfer data from the module to the Arduino or microcontroller. Additionally, it can be used for I2C communication as a clock pulse (SCL) or UART serial data transfer (TX).
3. **Pin 6 (MOSI):** This is the data input pin for the RFID module when using SPI communication.
4. **Pin 7 (SCK):** This pin is used to send the clock pulse required for SPI communication.

5. **Pin 8 (SS/SDA/RX):** The SS pin acts as the chip enable pin for SPI communication. It signals when the Arduino (master) must perform SPI communication. In I2C communication, this pin serves as the second pin (SDA). It is also used to receive data in UART communication (RX).

❖ NodeMCU:

The NodeMCU (Node MicroController Unit) is an open-source software and hardware development environment built around an inexpensive System-on-a-Chip (SoC) called the ESP8266. The ESP8266, designed and manufactured by Espressif Systems, contains the crucial elements of a computer: CPU, RAM, networking (WiFi), and even a modern operating system and SDK. That makes it an excellent choice for Internet of Things (IoT) projects of all kinds.

However, as a chip, the ESP8266 is also hard to access and use. You must solder wires, with the appropriate analog voltage, to its pins for the simplest tasks such as powering it on or sending a keystroke to the “computer” on the chip. You also have to program it in low-level machine instructions that can be interpreted by the chip hardware. This level of integration is not a problem using the ESP8266 as an embedded controller chip in mass-produced electronics. It is a huge burden for hobbyists, hackers, or students who want to experiment with it in their own IoT projects.



POWER pins There are four power pins. **VIN** pin and three **3.3V** pins.

- **VIN** can be used to directly supply the NodeMCU/ESP8266 and its peripherals. Power delivered on **VIN** is regulated through the onboard

regulator on the NodeMCU module – you can also supply 5V regulated to the **VIN** pin

- **3.3V** pins are the output of the onboard voltage regulator and can be used to supply power to external components.

GND are the ground pins of NodeMCU/ESP8266

I2C pins are used to connect I2C sensors and peripherals. Both I2C Master and I2C Slave are supported. I2C interface functionality can be realized programmatically, and the clock frequency is 100 kHz at a maximum. It should be noted that I2C clock frequency should be higher than the slowest clock frequency of the slave device.

GPIO pins NodeMCU/ESP8266 has 17 GPIO pins which can be assigned to functions such as I2C, I2S, UART, PWM, IR Remote Control, LED Light and Button programmatically. Each digital enabled GPIO can be configured to internal pull-up or pull-down, or set to high impedance. When configured as an input, it can also be set to edge-trigger or level-trigger to generate CPU interrupts.

ADC channel The NodeMCU is embedded with a 10-bit precision SAR ADC. The two functions can be implemented using ADC. Testing power supply voltage of VDD3P3 pin and testing input voltage of TOUT pin. However, they cannot be implemented at the same time.

UART pins NodeMCU/ESP8266 has 2 UART interfaces (UART0 and UART1) which provide asynchronous communication (RS232 and RS485), and can communicate at up to 4.5 Mbps. UART0 (TXD0, RXD0, RST0 & CTS0 pins) can be used for communication. However, UART1 (TXD1 pin) features only data transmit signal so, it is usually used for printing log.

SPI pins NodeMCU/ESP8266 features two SPIs (SPI and HSPI) in slave and master modes. These SPIs also support the following general-purpose SPI features:

- 4 timing modes of the SPI format transfer
- Up to 80 MHz and the divided clocks of 80 MHz
- Up to 64-Byte FIFO

SDIO pins NodeMCU/ESP8266 features Secure Digital Input/Output Interface (SDIO) which is used to directly interface SD cards. 4-bit 25 MHz SDIO v1.1 and 4-bit 50 MHz SDIO v2.0 are supported.

PWM pins The board has 4 channels of Pulse Width Modulation (PWM). The

PWM output can be implemented programmatically and used for driving digital motors and LEDs. PWM frequency range is adjustable from 1000 μ s to 10000 μ s (100 Hz and 1 kHz).

Control pins are used to control the NodeMCU/ESP8266. These pins include Chip Enable pin (EN), Reset pin (RST) and WAKE pin.

- **EN:** The ESP8266 chip is enabled when EN pin is pulled HIGH. When pulled LOW the chip works at minimum power.
- **RST:** RST pin is used to reset the ESP8266 chip.
- **WAKE:** Wake pin is used to wake the chip from deep-sleep.

Control Pins are used to control the NodeMCU/ESP8266. These pins include Chip Enable pin (EN), Reset pin (RST) and WAKE pin.

- **EN:** The ESP8266 chip is enabled when EN pin is pulled HIGH. When pulled LOW the chip works at minimum power.
- **RST:** RST pin is used to reset the ESP8266 chip.

WAKE: Wake pin is used to wake the chip from deep-sleep.

❖ Buzzer:

A buzzer is a small yet efficient component to add sound features to our project/system. It is very small and compact 2-pin structure hence can be easily used on breadboard, Perf Board and even on PCBs which makes this a widely used component in most electronic applications.



Buzzer Pin Configuration

1. **Pin 1 (Positive):** This pin is identified by the (+) symbol or the longer terminal lead. It can be powered by a 6V DC power source.
2. **Pin 2 (Negative):** This pin is identified by the shorter terminal lead and is typically connected to the ground of the circuit.

❖ Technical Specifications of ESP8266:

<u>MFRC522</u>	<u>NodeMCU ESP8266</u>
SDA	D4
SCK	D5
MOSI	D7
MISO	D6
IRQ	not required
GND	GND
RST	D0
3.3V	3V3

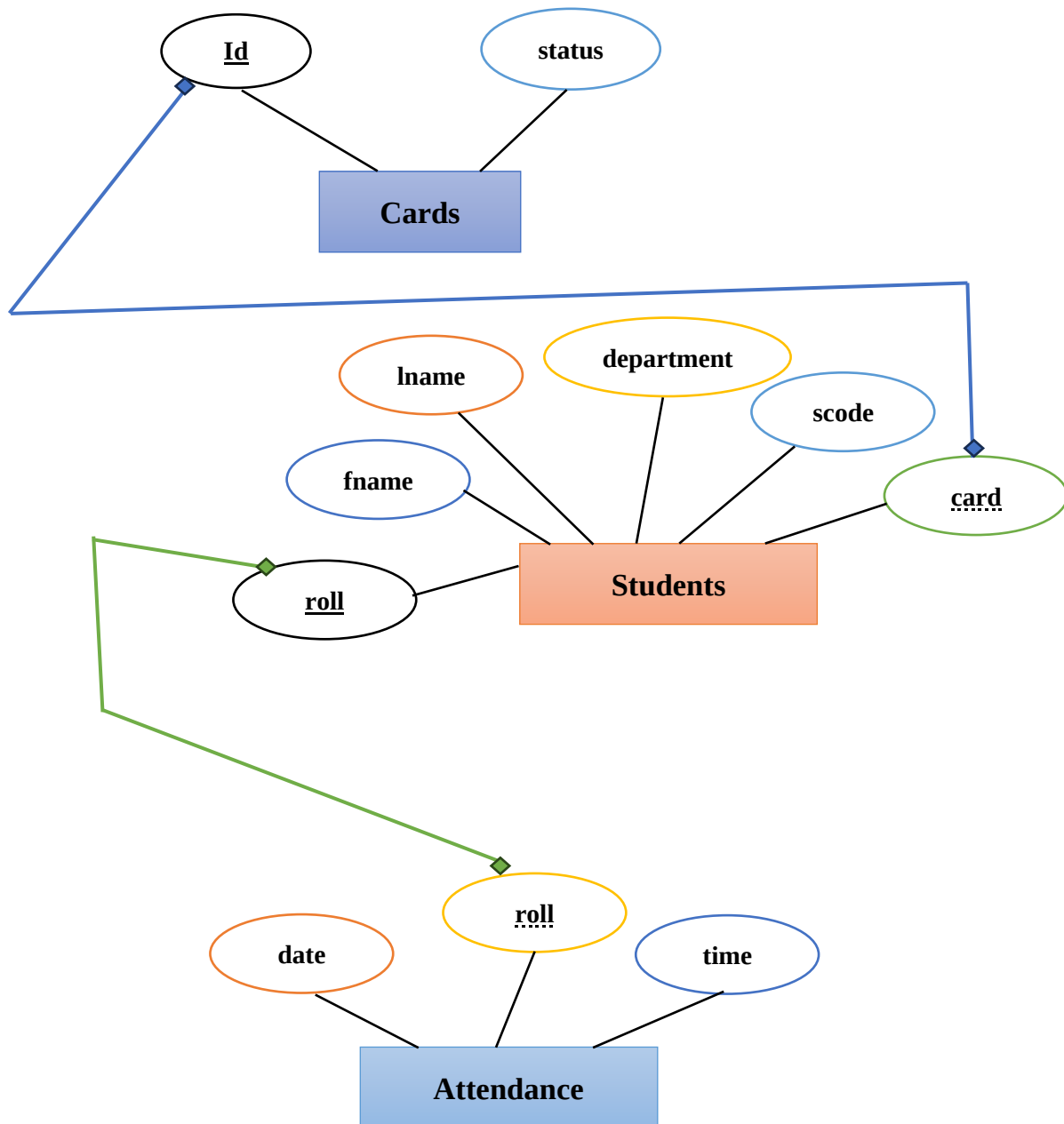
❖ Pin Mapping of LEDs Buzzer:

Component	Pin
Blue LED	D3
Red LED	D2
White LED	D1
Buzzer	D8

METHODOLOGY

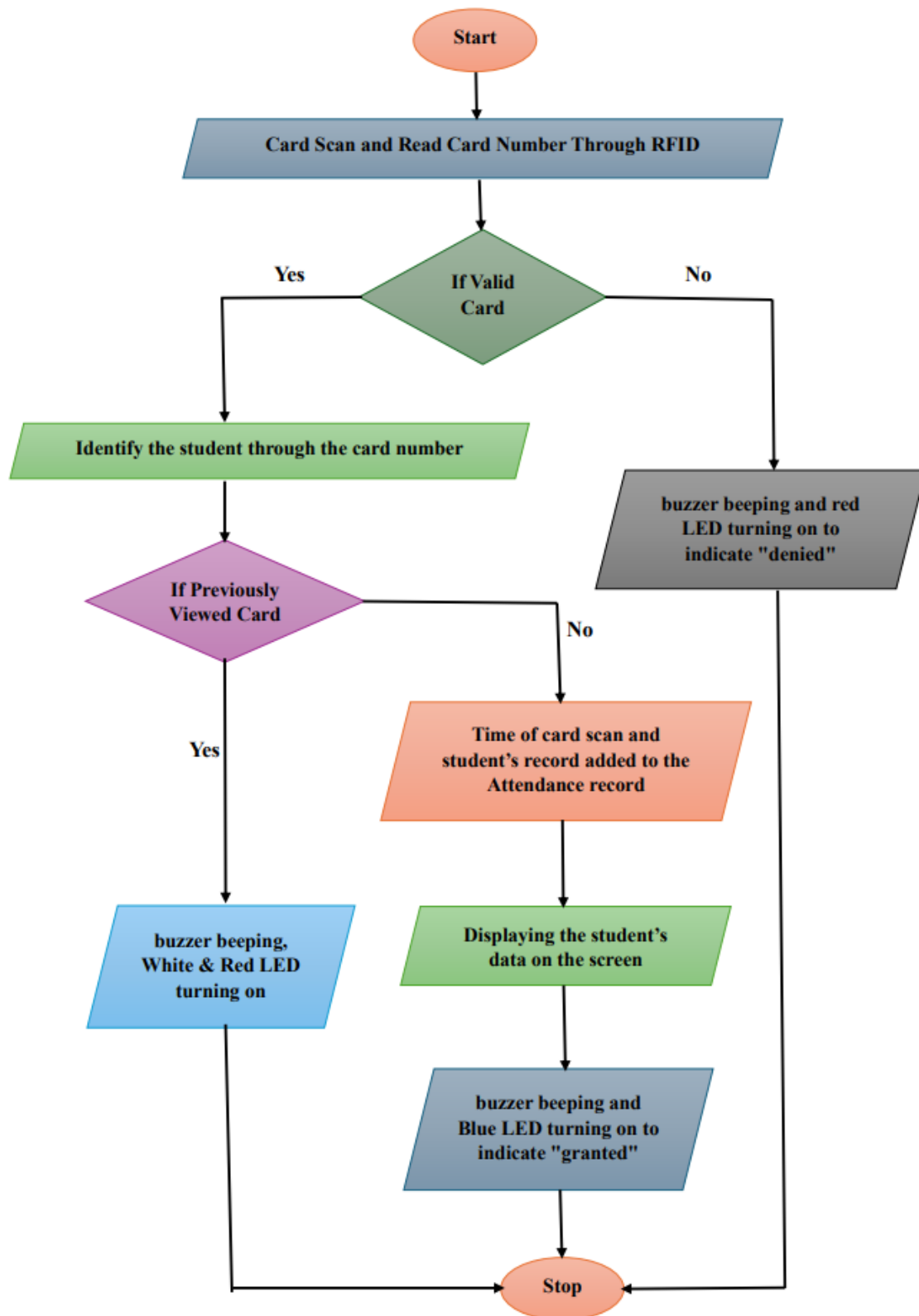
❖ PROBLEM FORMULATION:

➤ ER Diagram (Entity Relationship Diagram) :



❖ ALGORITHM DESCRIPTION:

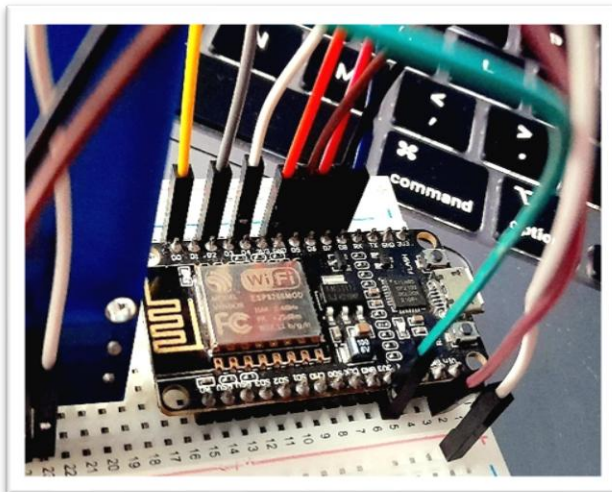
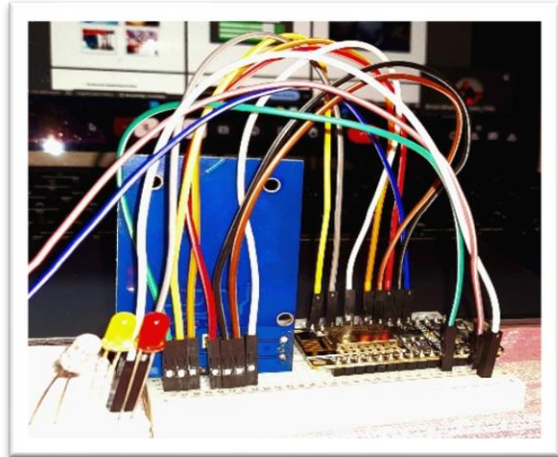
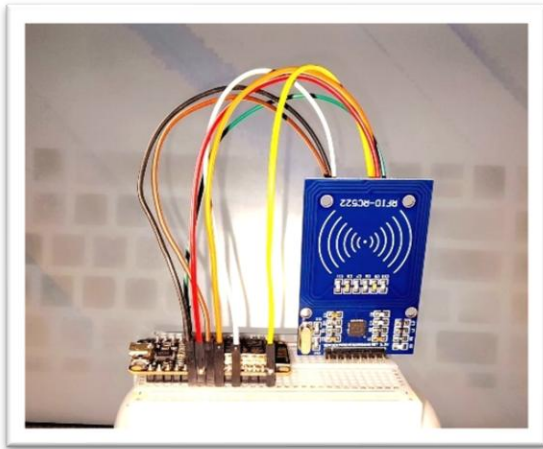
➤ Flow diagram:



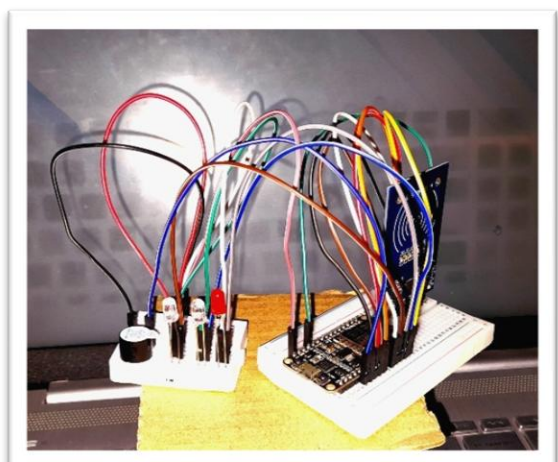
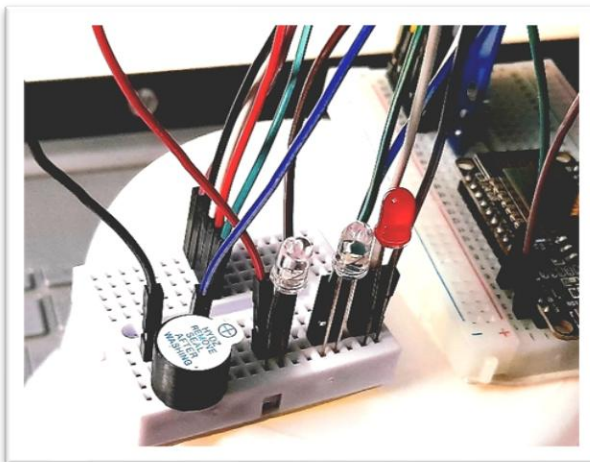
IMPLEMENTATION

➤ Hardware implementation :

○ RFID connection with NodeMCU:

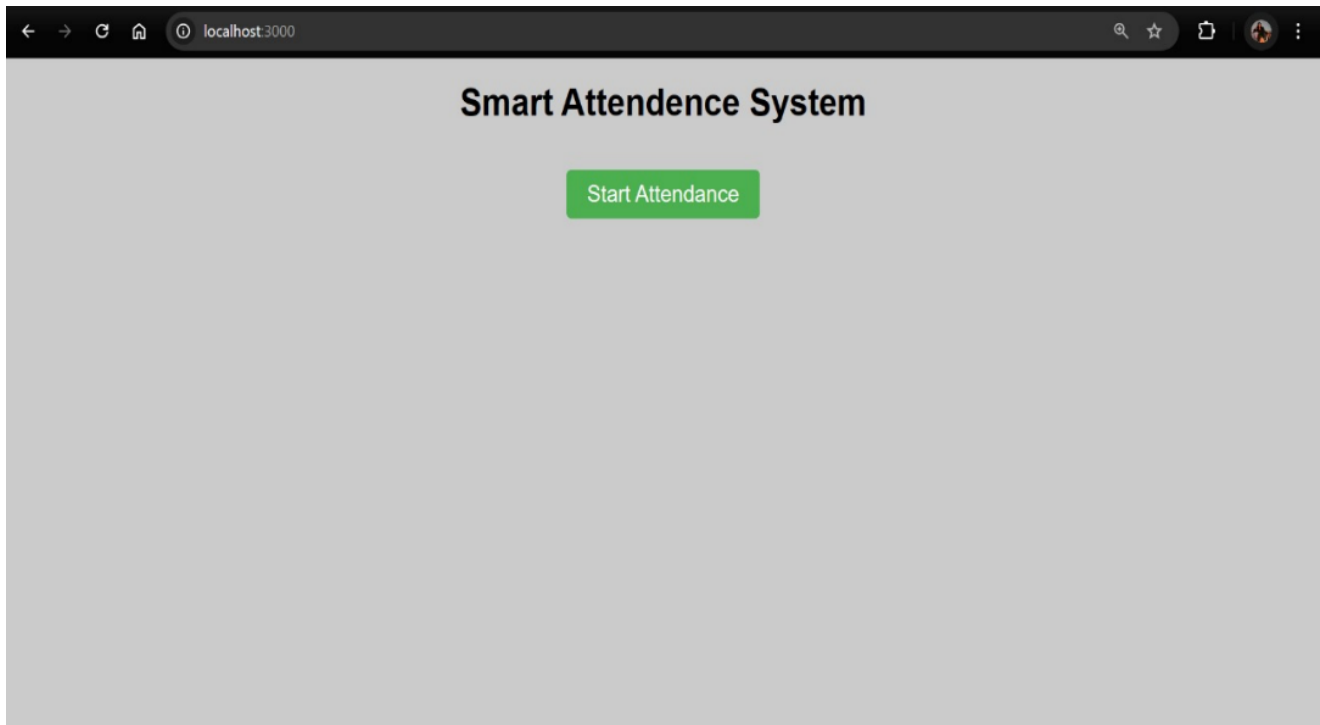


○ Buzzer and LED connection:

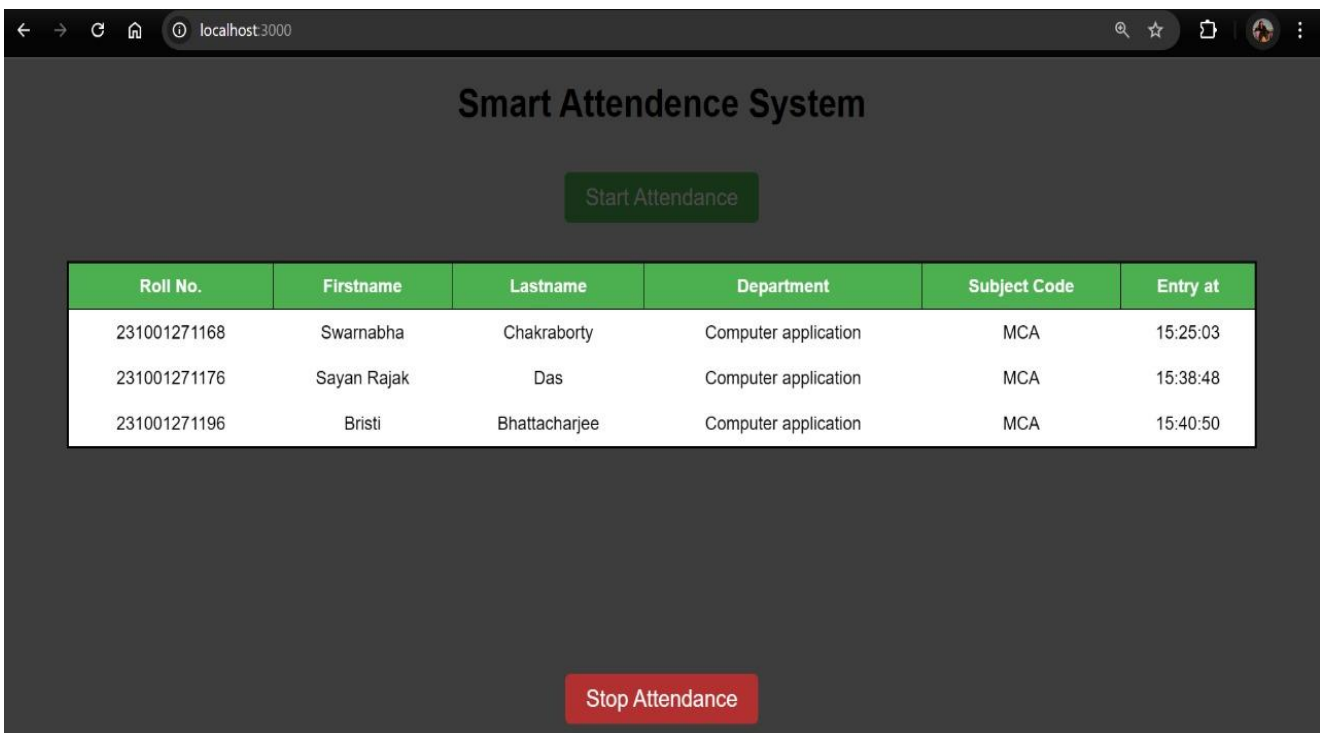


➤ Software implementation

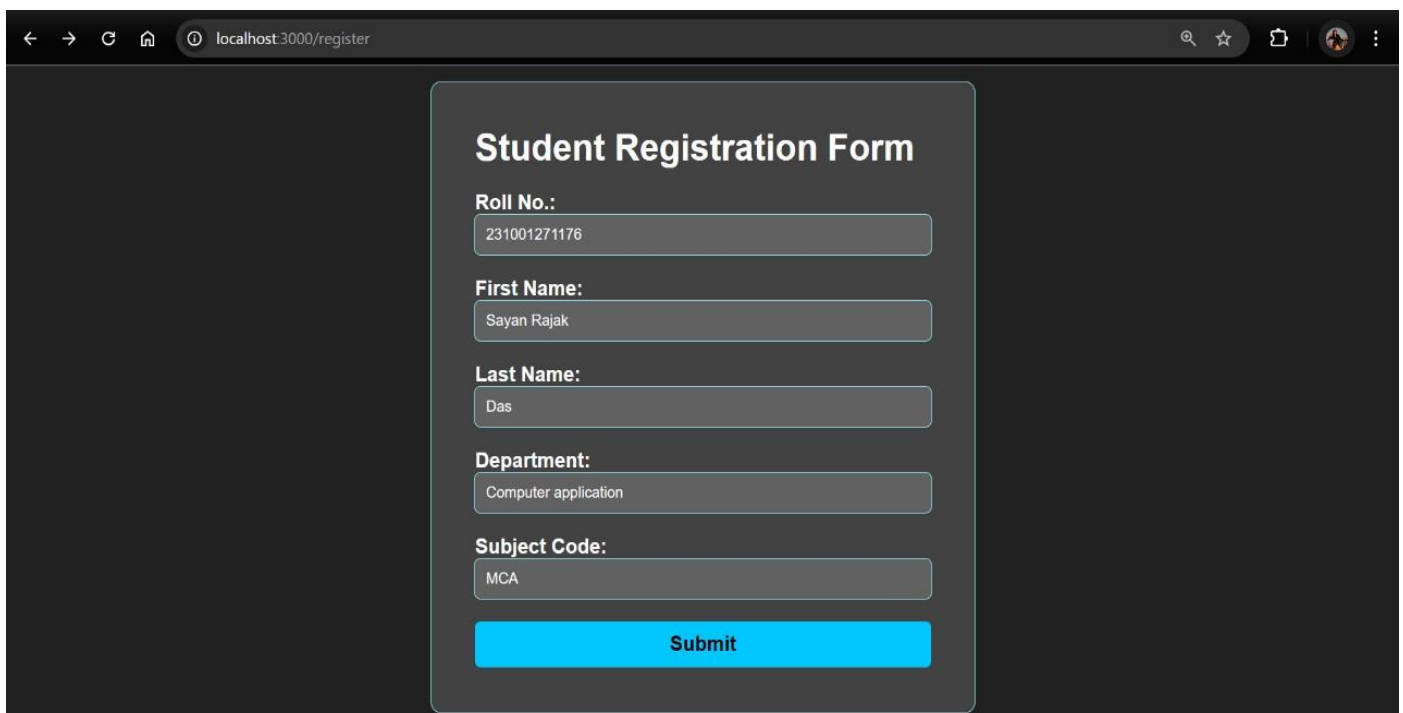
- Home page:



- Attendance page:



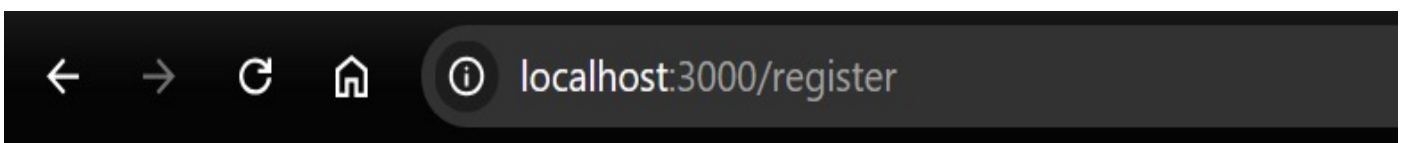
- Registration page:



The screenshot shows a web browser window with the address bar displaying "localhost:3000/register". The page features a "Student Registration Form" with the following fields and values:

Field	Value
Roll No.:	231001271176
First Name:	Sayan Rajak
Last Name:	Das
Department:	Computer application
Subject Code:	MCA

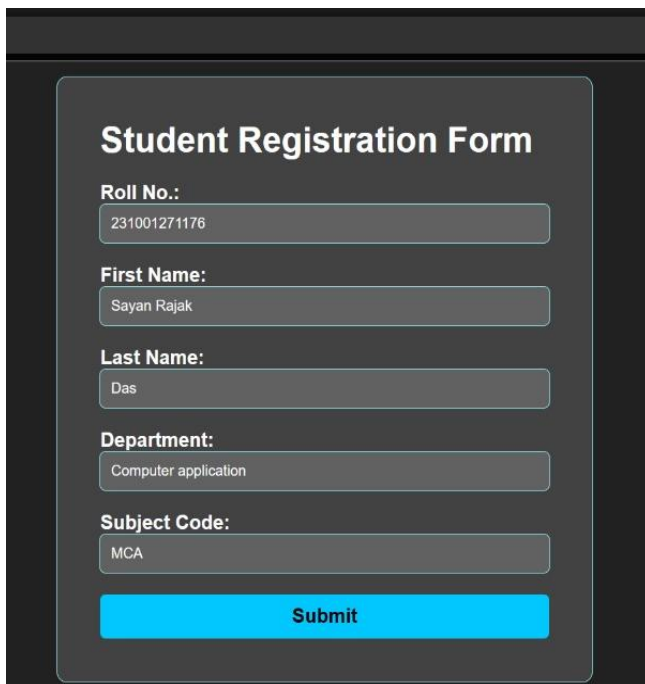
A blue "Submit" button is located at the bottom of the form.



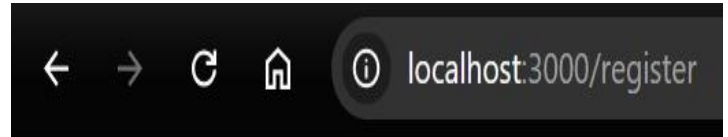
Card Sayan Rajak Das added.

RESULT AND DISCUSSION

1. Student registration:

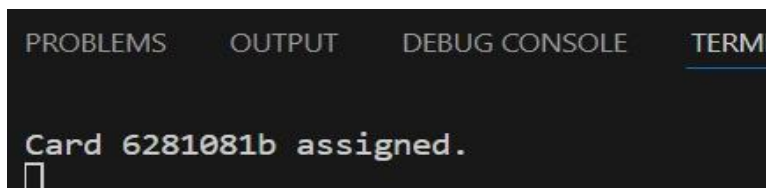


The screenshot shows a web form titled "Student Registration Form". It contains several input fields: "Roll No." with the value "231001271176", "First Name" with "Sayan Rajak", "Last Name" with "Das", "Department" with "Computer application", and "Subject Code" with "MCA". A red "Submit" button is at the bottom.



Card Sayan Rajak Das added.

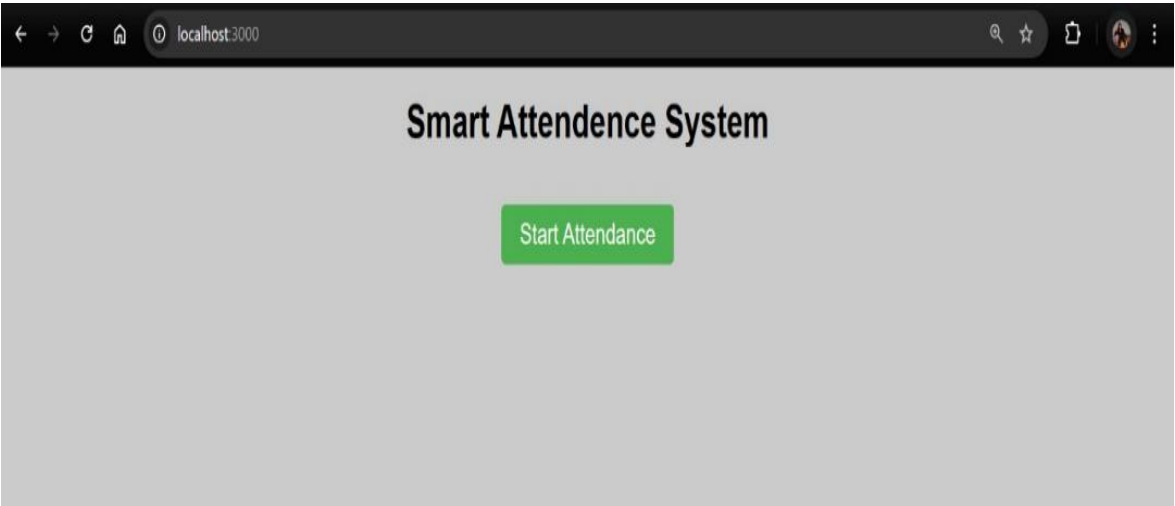
2. A card will be assigned to the student:



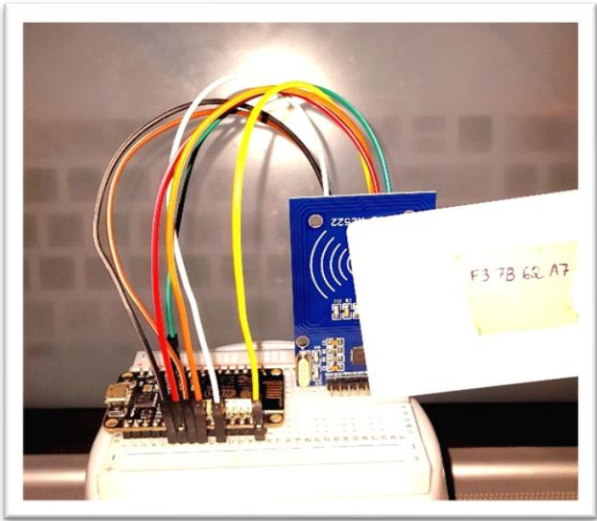
3. Student's data added to the JSON file:



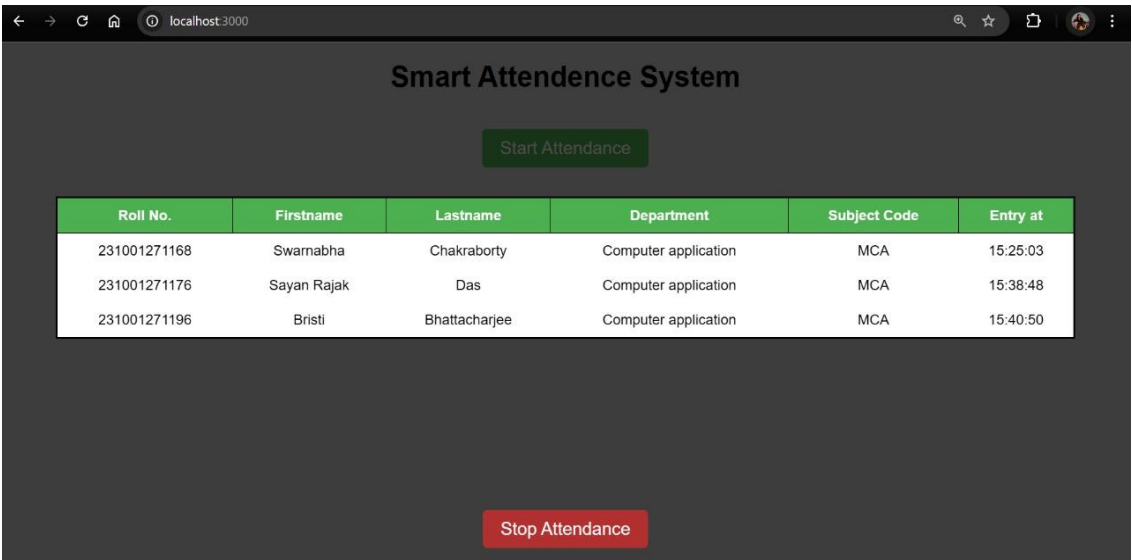
4. Open the Homepage to start attendance:



5. Read the RFID card number using RFID scanner:



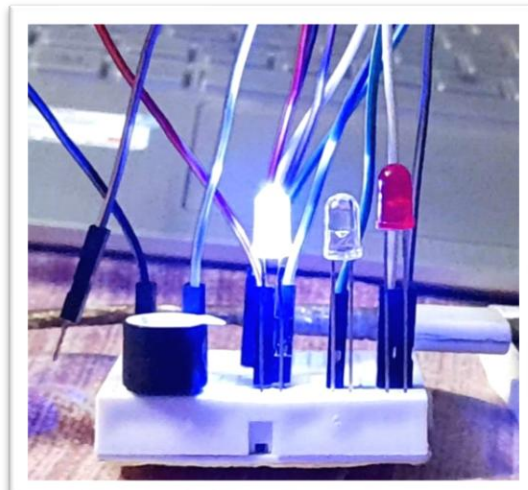
6. If the card is valid and the card is scanned for the first time Student's details displayed on the attendance page:



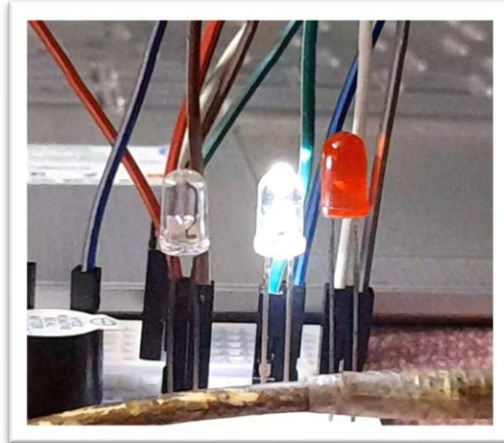
7. Attendance is recorded in the JSON file:

```
{ } attendance.json X
{ } attendance.json > { } 0 > [ ] Record > { } 1
1 [{"Date": "2025-01-16", "Record": [
2   {
3     "roll": "231001271168",
4     "fname": "Swarnabha",
5     "lname": "Chakraborty",
6     "department": "Computer application",
7     "scode": "MCA",
8     "time": "15:25:03"
9   },
10  {
11    "roll": "231001271176",
12    "fname": "Sayan Rajak",
13    "lname": "Das",
14    "department": "Computer application",
15    "scode": "MCA",
16    "time": "15:38:48"
17  },
18  {
19    "roll": "231001271196",
20    "fname": "Bristi",
21    "lname": "Bhattacharjee",
22    "department": "Computer application",
23    "scode": "MCA",
24    "time": "15:40:50"
25  }
26  ]}]
```

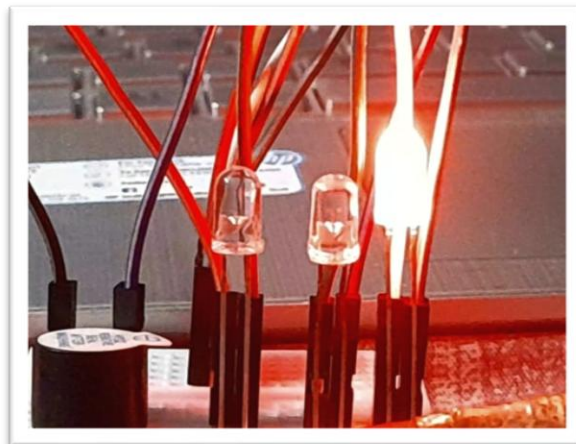
8. Buzzer beeping and Blue LED turning on to indicate "granted" :



- If the card is valid but previously viewed then buzzer will beep and White LED will turn on:



- If the card is not valid then buzzer will beep and red LED will turn on to indicate "denied":



Future Scope

The future of the smart attendance system offers exciting opportunities for enhancement:

1. **Biometric and AI Integration:** Future systems could integrate facial recognition or AI to further improve accuracy and reduce reliance on RFID cards.
2. **Mobile App Integration:** Developing user-friendly mobile applications for attendance tracking, notifications, and personalized attendance reports.
3. **Scalability for Multiple Applications:** Expanding the system's use to conferences, offices, or events to provide universal attendance solutions.
4. **Cloud and Data Analytics:** Utilizing cloud platforms and advanced analytics to monitor trends, generate predictive insights, and optimize resource allocation.
5. **IoT and Smart Environments:** Integrating with broader IoT ecosystems for enhanced functionality, such as access control or resource management in smart buildings.

Conclusion

The development of a smart attendance management system using RFID technology offers a transformative solution to the challenges of traditional attendance tracking. By automating the process, the system eliminates manual record-keeping, reduces errors, and saves time for both educators and students. Faculty members are freed from tedious administrative tasks, while students benefit from a transparent system that provides real-time access to attendance data and ensures accountability.

The integration of a central database enables real-time data availability, allowing institutions to monitor trends, track absenteeism, and generate actionable insights for improved decision-making. The system's eco-friendly approach, reducing reliance on paper-based records, and its features like notifications and mobile app access enhance user convenience while promoting sustainability.

RFID technology ensures high security and accuracy, eliminating duplication risks and maintaining the integrity of records. Additionally, the system's scalability and versatility make it adaptable for broader applications in workplaces, events, and other domains.

In summary, the smart attendance management system revolutionizes traditional methods, combining automation, accuracy, and sustainability to enhance efficiency and transparency. With its potential for future integration with advanced technologies, the system sets a foundation for smarter, more efficient attendance management in educational institutions and beyond.

References

1. [U. Farooq, M. ul Hasan, M. Amar, A. Hanif and M. Usman Asad, "RFID Based Security and Access Control System" IACSIT International Journal of Engineering and Technology, vol. 6, pp. 309- 314.](#)
2. <https://azure.microsoft.com/en-in/overview/internet-of-things-iot/iot-technology-protocols/>
3. <https://randomnerdtutorials.com/esp8266-pinout-reference-gpios/>
4. <https://www.w3schools.com/nodejs/>
5. <https://www.tutorialspoint.com/>
6. <https://www.geeksforgeeks.org/>

Appendix

Arduino Source code:

```
#ifndef ESP32
#include <WiFi.h>
#include <HTTPClient.h>
#else
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClient.h>
#endif
#include <SPI.h>
#include <MFRC522.h>

const char* ssid = "Bristi"; // wifi name
const char* password = "0192837465"; // wifi password
WiFiClient wifiClient;
String serverName = "http://192.168.152.174:3000/addattendance"; // put ip address

#define SS_PIN D4
#define RST_PIN D0
MFRC522 rfid(SS_PIN, RST_PIN); // Instance of the class
MFRC522::MIFARE_Key key;
// Init array that will store new NUID
byte nuidPICC[4];
void setup() {
  pinMode(D1, OUTPUT); // already present
  pinMode(D2, OUTPUT); // denied
  pinMode(D3, OUTPUT); // granted
  pinMode(D8, OUTPUT); // Buzzer
  Serial.begin(9600);
  SPI.begin(); // Init SPI bus
  WiFi.begin(ssid, password);
  Serial.println("Connecting");
  Serial.println(WiFi.status());
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.print("Connected to WiFi network with IP Address: ");
  Serial.println(WiFi.localIP());

  rfid.PCD_Init(); // Init MFRC522
  Serial.println();
  Serial.print(F("Reader :"));
  rfid.PCD_DumpVersionToSerial();
  for (byte i = 0; i < 6; i++) {
    key.keyByte[i] = 0xFF;
  }
  Serial.println();
  Serial.println(F("This code scan the MIFARE Classic NUID."));
  Serial.print(F("Using the following key:"));
  printHex(key.keyByte, MFRC522::MF_KEY_SIZE);
}

void loop() {
  // Reset the loop if no new card present on the sensor/reader. This saves the entire process when idle.
```

```

if (!rfid.PICC_IsNewCardPresent())
    return;
// Verify if the NUID has been readed
if (!rfid.PICC_ReadCardSerial())
    return;
Serial.print(F("PICC type: "));
MFRC522::PICC_Type piccType = rfid.PICC_GetType(rfid.uid.sak);
Serial.println(rfid.PICC_GetTypeName(piccType));
// Check is the PICC of Classic MIFARE type
if (piccType != MFRC522::PICC_TYPE_MIFARE_MINI && piccType != MFRC522::PICC_TYPE_MIFARE_1K
&& piccType != MFRC522::PICC_TYPE_MIFARE_4K) {
    Serial.println(F("Your tag is not of type MIFARE Classic."));
    return;
}
if (rfid.uid.uidByte[0] != nuidPICC[0] || rfid.uid.uidByte[1] != nuidPICC[1] || rfid.uid.uidByte[2] != nuidPICC[2] ||
rfid.uid.uidByte[3] != nuidPICC[3]) {
    Serial.println(F("A new card has been detected."));
    // Store NUID into nuidPICC array
    for (byte i = 0; i < 4; i++) {
        nuidPICC[i] = rfid.uid.uidByte[i];
    }
    Serial.println(F("The NUID tag is:"));
    Serial.print(F("In hex: "));
    printHex(rfid.uid.uidByte, rfid.uid.size);

    String uid = "";
    for (byte i = 0; i < rfid.uid.size; i++) {
        uid += String(rfid.uid.uidByte[i], HEX);
    }

    Serial.println("RFID UID: " + uid);
    sendToNodeJS(uid);

    // Serial.print(F("In dec: "));
    // printDec(rfid.uid.uidByte, rfid.uid.size);
    // Serial.println();
} else Serial.println(F("Card read previously."));
Serial.println(F("The NUID tag is:"));
Serial.print(F("In hex: "));
printHex(rfid.uid.uidByte, rfid.uid.size);

String uid = "";
for (byte i = 0; i < rfid.uid.size; i++) {
    uid += String(rfid.uid.uidByte[i], HEX);
}

Serial.println("RFID UID: " + uid);
//sendToNodeJS(uid);
digitalWrite(D1, HIGH);
digitalWrite(D8, HIGH);
delay(300);
digitalWrite(D1, LOW);
digitalWrite(D8, LOW);
delay(300);
digitalWrite(D2, HIGH);
digitalWrite(D8, HIGH);
delay(300);
digitalWrite(D2, LOW);

```

```

digitalWrite(D8, LOW);
delay(300);
digitalWrite(D1, HIGH);
digitalWrite(D8, HIGH);
delay(300);
digitalWrite(D1, LOW);
digitalWrite(D8, LOW);
delay(300);
digitalWrite(D2, HIGH);
digitalWrite(D8, HIGH);
delay(300);
digitalWrite(D2, LOW);
digitalWrite(D8, LOW);
delay(300);

// Halt PICC
rfid.PICC_HaltA();
//Stop encryption on PCD
rfid.PCD_StopCrypto1();
}
/**
  Helper routine to dump a byte array as hex values to Serial.
*/
void printHex(byte* buffer, byte bufferSize) {
  for (byte i = 0; i < bufferSize; i++) {
    Serial.print(buffer[i] < 0x10 ? " 0" : " ");
    Serial.print(buffer[i], HEX);
  }
}
/**
  Helper routine to dump a byte array as dec values to Serial.
*/
void printDec(byte* buffer, byte bufferSize) {
  for (byte i = 0; i < bufferSize; i++) {
    Serial.print(buffer[i] < 0x10 ? " 0" : " ");
    Serial.print(buffer[i], DEC);
  }
}

void sendToNodeJS(String uid) {
  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;
    http.begin(wifiClient, serverName);
    http.addHeader("Content-Type", "application/x-www-form-urlencoded");
    String httpRequestData = "&rfid="+ uid + "&status=approved";
    int httpCode = http.POST(httpRequestData);
    Serial.println(http.getString());

    if(http.getString() == "denied") {
      digitalWrite(D2, HIGH);
      digitalWrite(D8, HIGH);
      delay(300);
      digitalWrite(D2, LOW);
      digitalWrite(D8, LOW);
      delay(300);
      digitalWrite(D2, HIGH);
      digitalWrite(D8, HIGH);
      delay(300);
    }
  }
}

```

```

digitalWrite(D2, LOW);
digitalWrite(D8, LOW);
delay(300);
digitalWrite(D2, HIGH);
digitalWrite(D8, HIGH);
delay(300);
digitalWrite(D2, LOW);
digitalWrite(D8, LOW);
delay(300);
digitalWrite(D2, HIGH);
digitalWrite(D8, HIGH);
delay(300);
digitalWrite(D2, LOW);
digitalWrite(D8, LOW);
delay(300);
} else if(http.getString() == "granted") {
  digitalWrite(D3, HIGH);
  digitalWrite(D8, HIGH);
  delay(1000);
  digitalWrite(D3, LOW);
  digitalWrite(D8, LOW);
  delay(1000);
  // digitalWrite(D3, HIGH);
  // delay(300);
  // digitalWrite(D3, LOW);
  // delay(1000);
} else {
  digitalWrite(D1, HIGH);
  digitalWrite(D8, HIGH);
  delay(300);
  digitalWrite(D1, LOW);
  digitalWrite(D8, LOW);
  delay(300);
  digitalWrite(D1, HIGH);
  digitalWrite(D8, HIGH);
  delay(300);
  digitalWrite(D1, LOW);
  digitalWrite(D8, LOW);
  delay(300);
}
Serial.print("HTTP Response code: ");
Serial.println(httpCode);
// Free resources
http.end();
} else {
  Serial.println("WiFi Disconnected");
}
}

```

Node js Source code:

index.js

```

const express = require('express');
const fs = require('fs')
const app = express();
const ejs = require('ejs');
const cors = require('cors');

```

```

function chooseCardId() {

```

```

const cards = JSON.parse(fs.readFileSync('card.json'));

// filter out cards where status is "assigned"
const unassignedCards = cards.filter(card => card.status !== 'assigned');

if (unassignedCards.length === 0) {
  return -1;
}
// choose a random card from the unassigned cards
const randomIndex = Math.floor(Math.random() * unassignedCards.length);
const randomCard = unassignedCards[randomIndex];

// return the chosen card ID
return randomCard.id;
}

function assignCard(cardId) {
  const cards = JSON.parse(fs.readFileSync('card.json'));

  // find the index of the card with the given ID
  const cardIndex = cards.findIndex(card => card.id === cardId);

  // if the card was found, update its status to "assigned"
  if (cardIndex !== -1) {
    cards[cardIndex].status = 'assigned';
    fs.writeFileSync('card.json', JSON.stringify(cards));
    console.log(`Card ${cardId} assigned.`);
  } else {
    console.log(`Card ${cardId} not found.`);
  }
}

function getCurrentTime() {
  const now = new Date();
  const hours = now.getHours().toString().padStart(2, '0');
  const minutes = now.getMinutes().toString().padStart(2, '0');
  const seconds = now.getSeconds().toString().padStart(2, '0');
  return `${hours}:${minutes}:${seconds}`;
}

function getCurrentDate() {
  const now = new Date();
  const year = now.getFullYear().toString().padStart(4, '0');
  const month = (now.getMonth() + 1).toString().padStart(2, '0');
  const day = now.getDate().toString().padStart(2, '0');
  return `${year}-${month}-${day}`;
}

app.use(express.json())
app.use(express.urlencoded({ extended: false }))
app.set('view engine', 'ejs');
app.use(cors());

app.get('/', (req, res) => {
  const name = 'John';
  res.render('index');
});

```

```

app.get('/dailyreport', (req, res) => {
  res.render('report', {
    title: "form"
  })
})

app.post('/dailyreport', (req, res) => {
  let currentDate = req.body.date
  const attendance = JSON.parse(fs.readFileSync('attendance.json'));
  const AttendanceRecord = attendance.filter(record => record.Date === currentDate);
  if (AttendanceRecord.length > 0) {
    // res.json(AttendanceRecord);
    res.render('report', {
      title: "Attendance Data",
      date: currentDate,
      attendance: AttendanceRecord[0].Record
    })
  } else {
    res.render('report', {
      title: "Attendance Data",
      date: currentDate,
      attendance: '404 not found'
    })
  }
})

app.get('/register', (req, res) => {
  res.render('studentRegister')
})

app.post('/register', (req, res) => {
  let student = req.body;
  let studentRecord = {
    "roll": student.roll,
    "fname": student.fname,
    "lname": student.lname,
    "department": student.department,
    "scode": student.scode,
    "card": chooseCardId()
  }

  if (studentRecord.card === -1) {
    res.status(404).send("nahi jagah hai")
  } else {
    assignCard(studentRecord.card)
    const cards = JSON.parse(fs.readFileSync('students.json'));
    cards.push(studentRecord);
    fs.writeFileSync('students.json', JSON.stringify(cards));
    res.send(` Card ${studentRecord.fname} ${studentRecord.lname} added.`)
  }
})

app.post('/', (req, res) => {
  console.log(req.body)
  console.log(req.body.rfid)

```



```

let id = req.body.rfid;
let card = {
  "id": id,
  "status": "notassigned"
}
const cards = JSON.parse(fs.readFileSync('card.json'));
const recordedCards = cards.filter(card => card.id === id);
if (recordedCards.length === 0) {
  cards.push(card);
  fs.writeFileSync('card.json', JSON.stringify(cards));
  res.send(`Card ${id} recorded.`)
} else {
  res.send("Card Already Added")
}

/* fs.writeFile('database.json', JSON.stringify(req.body), (err) => {
  if (err) {
    console.error(err);
    res.status(500).send('Error writing JSON data to file');
  } else {
    console.log('JSON data written to file');
    res.status(200).send('JSON data written to file');
  }
}); */
})

app.post('/startattendance', (req, res) => {
  /* const attendancerecord = {
    "Date": getCurrentDate(),
    "Record": []
  }
  const records = JSON.parse(fs.readFileSync('attendance.json'));
  records.push(attendancerecord);
  fs.writeFileSync('attendance.json', JSON.stringify(records));
  res.send('attendance started') */
  const currentDate = getCurrentDate();
  const attendance = JSON.parse(fs.readFileSync('attendance.json'));
  const currentAttendanceRecord = attendance.find(record => record.Date === currentDate);
  if (currentAttendanceRecord) {
    res.send('Attendance for today has already been started');
  } else {
    const newAttendanceRecord = {
      "Date": currentDate,
      "Record": []
    }
    attendance.push(newAttendanceRecord);
    fs.writeFileSync('attendance.json', JSON.stringify(attendance));
    res.send('Attendance for today has been started');
  }
})

app.post('/addattendance', (req, res) => {
  const students = JSON.parse(fs.readFileSync('students.json'));
  const student = students.filter(student => student.card === req.body.rfid)
  if (Object.keys(student).length === 0) {
    res.send("denied");
  } else {
    console.log(student)
  }
})

```

```

const attendancerecord = {
  "roll": student[0].roll,
  "fname": student[0].fname,
  "lname": student[0].lname,
  "department": student[0].department,
  "scode": student[0].scode,
  "time": getcurrentTime()
}

const attendance = JSON.parse(fs.readFileSync('attendance.json'));
const recordtoday = attendance.filter(attendance => attendance.Date === getcurrentTime())
console.log(recordtoday)
// recordtoday[0].Record.push(attendancerecord);
// fs.writeFileSync('attendance.json', JSON.stringify(recordtoday));
// res.send("attendance recorder");
const studentRecordExists = recordtoday[0].Record.some(record => record.roll === student[0].roll);
if (studentRecordExists) {
  res.send(`${student[0].fname} ${student[0].lname} has already been marked present for today`);
} else {
  recordtoday[0].Record.push(attendancerecord);
  fs.writeFileSync('attendance.json', JSON.stringify(attendance));
  res.send("granted");
}
}
})

app.get('/addattendance', (req, res) => {
  const attendance = JSON.parse(fs.readFileSync('attendance.json'));
  const recordtoday = attendance.filter(attendance => attendance.Date === getcurrentTime());
  res.send(recordtoday);
})

app.listen(3000, () => {
  console.log(`http://192.168.152.174:3000`);
})

```

cards.json

```

[{"id":"f3265ea7","status":"assigned"}, {"id":"b38ca433","status":"assigned"}, {"id":"f37b62a7","status":"notassigned"}, {"id":"628181b","status":"assigned"}, {"id":"a04de1b","status":"assigned"}]

```

students.json

```

[{"roll":"231001271168","fname":"Swarnabha","lname":"Chakraborty","department":"Computer application","scode":"MCA","card":"f3265ea7"}, {"roll":"231001271196","fname":"Bristi","lname":"Bhattacharjee","department":"Computer application","scode":"MCA","card":"b38ca433"}, {"roll":"231001271176","fname":"Sayan Rajak","lname":"Das","department":"Computer application","scode":"MCA","card":"628181b"}, {"roll":"231001271026","fname":"Labani","lname":"Das","department":"Computer application","scode":"MCA","card":"a04de1b"}]

```

attendance.json

```

[{"Date":"2025-01-16","Record":[{"roll":"231001271168","fname":"Swarnabha","lname":"Chakraborty","department":"Computer application","scode":"MCA","time":"15:25:03"}, {"roll":"231001271176","fname":"Sayan Rajak","lname":"Das","department":"Computer application","scode":"MCA","time":"15:38:48"}, {"roll":"231001271196","fname":"Bristi","lname":"Bhattacharjee","department":"Computer application","scode":"MCA","time":"15:40:50"}]}, {"Date":"2025-01-16"}]

```

```
17","Record":[{"roll":"231001271196","fname":"Bristi","lname":"Bhattacharjee","department":"Computer application","scode":"MCA","time":"11:35:12"},{"roll":"231001271168","fname":"Swarnabha","lname":"Chakraborty","department":"Computer application","scode":"MCA","time":"13:24:44"},{"roll":"231001271176","fname":"Sayant Rajak","lname":"Das","department":"Computer application","scode":"MCA","time":"13:44:11"},{"roll":"231001271026","fname":"Labani","lname":"Das","department":"Computer application","scode":"MCA","time":"13:46:11"}]]]
```

Front end Source code: index.ejs

```
<!DOCTYPE html>
<html>

<head>
  <title>Smart Attendance System</title>
  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
      font-family: Arial, Helvetica, sans-serif;
    }

    body {
      background-color: rgb(203, 203, 203);
    }

    h1 {
      text-align: center;
      padding: 20px;
    }

    table {
      width: 80%;
      margin: auto;
      border-collapse: collapse;
      border: 2px solid black;
      text-align: center;
    }

    th,
    td {
      padding: 10px;
      border: 1px solid black;
    }

    #btn {
      display: block;
      margin: 20px auto;
      padding: 10px 20px;
      font-size: 20px;
      background-color: #4CAF50;
      color: white;
      border: none;
      border-radius: 5px;
    }
  </style>
</head>

<body>
  <div>
    <h1>Smart Attendance System</h1>
    <table>
      <tr>
        <th>Roll No</th>
        <th>Name</th>
        <th>Department</th>
        <th>Scode</th>
        <th>Time</th>
      </tr>
      <tr>
        <td>231001271196</td>
        <td>Bristi</td>
        <td>Bhattacharjee</td>
        <td>MCA</td>
        <td>11:35:12</td>
      </tr>
      <tr>
        <td>231001271168</td>
        <td>Swarnabha</td>
        <td>Chakraborty</td>
        <td>MCA</td>
        <td>13:24:44</td>
      </tr>
      <tr>
        <td>231001271176</td>
        <td>Sayant Rajak</td>
        <td>Das</td>
        <td>MCA</td>
        <td>13:44:11</td>
      </tr>
      <tr>
        <td>231001271026</td>
        <td>Labani</td>
        <td>Das</td>
        <td>MCA</td>
        <td>13:46:11</td>
      </tr>
    </table>
    <div>
      <button>Add Record</button>
    </div>
  </div>
</body>
</html>
```

```

    cursor: pointer;
}

.mainAttendanceModal {
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background-color: rgba(0, 0, 0, 0.7);
    display: none;
    flex-direction: column;
    justify-content: center;
    align-items: center;
}

.mainAttendanceModal table {
    width: 90%;
    margin: auto;
    background-color: white;
}

.mainAttendanceModal th {
    background-color: #4CAF50;
    color: white;
}

.mainAttendanceModal td {
    border: none;
}
</style>
</head>

<body>
<h1 class="label">Smart Attendance System</h1>
<div class="mainAttendanceModal">
    <table id="studentRecords">
        <tr>
            <th>Roll No.</th>
            <th>Firstname</th>
            <th>Lastname</th>
            <th>Department</th>
            <th>Subject Code</th>
            <th>Entry at</th>
        </tr>
        <!-- <tr>
            <td>S-417</td>
            <td>Ayush</td>
            <td>Singh</td>
            <td>Computer Science</td>
            <td>CMSA</td>
            <td>12:09:23</td>
        </tr> -->
    </table>
    <button id="btn" class="btn1" onclick="stopAttendance()" style="background-color: #b33030 !important;">Stop
        Attendance</button>
</div>
<button id="btn" class="btn2" onclick="startAttendance()">Start Attendance</button>

```

```

<script>
  let mainAttendanceModal = document.querySelector('.mainAttendanceModal');
  var fetchInterval;
  let table = document.getElementById('studentRecords');
  const startAttendance = () => {
    mainAttendanceModal.style.display = "flex";

    // start Attendance
    // getAttendance();
    fetch('http://localhost:3000/startattendance', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      // body: JSON.stringify(data)
    })
    .then(response => response.text())
    .then(data => {
      getAttendance();
    })
    .catch((error) => {
      console.error('Error:', error);
    });
  }

  const stopAttendance = () => {
    document.querySelector('.label').style = "display : none";
    mainAttendanceModal.style = "position: relative;";
    mainAttendanceModal.style = "display: flex;";
    document.querySelector('.btn1').style = "display : none";
    document.querySelector('.btn2').style = "display : none";
    clearInterval(fetchInterval);
  }

  const getAttendance = () => {
    var prevlength = 0;
    fetchInterval = setInterval(() => {
      fetch('http://localhost:3000/addattendance', {
        method: 'GET',
        headers: {
          'Content-Type': 'application/json'
        },
        // body: JSON.stringify(data)
      })
      .then(response => response.json())
      .then(data => {
        //
        if (data.length > 0) {
          if (prevlength === 0 || data[0].Record.length > prevlength) {
            deleteRowOfTable();
            console.log(data[0].Record)
            insertDataToTable(data[0].Record);
            prevlength = data[0].Record.length;
          }
        }
      })
      .catch((error) => {
        console.error('Error:', error);
      });
    }, 1000);
  }

```

```

    });
    }, 100)

}

const insertDataToTable = (data) => {
  data.forEach(element => {
    let newRow = table.insertRow();
    let cell1 = newRow.insertCell(0);
    let cell2 = newRow.insertCell(1);
    let cell3 = newRow.insertCell(2);
    let cell4 = newRow.insertCell(3);
    let cell5 = newRow.insertCell(4);
    let cell6 = newRow.insertCell(5);
    cell1.innerHTML = element.roll;
    cell2.innerHTML = element.fname;
    cell3.innerHTML = element.lname;
    cell4.innerHTML = element.department;
    cell5.innerHTML = element.scode;
    cell6.innerHTML = element.time;
  });
}

const deleteRowOfTable = () => {
  let rowCount = table.rows.length;
  for (let i = rowCount - 1; i >= 1; i--) {
    table.deleteRow(i);
  }
}

</script>
</body>

</html>

```

report.ejs

```

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Daily Attendance</title>
  <style>
    body {
      background-color: #fff;
      font-family: Arial, Helvetica, sans-serif;
    }

    form {
      display: flex;
      flex-direction: column;
      align-items: center;
    }

    .tableDate {

```

```

display: flex;
flex-direction: row;
align-items: center;
justify-content: flex-end;
gap: 10px;
}

.tableDate input {
margin: 0 !important;
}

label {
margin-bottom: 10px;
}

input[type="date"] {
padding: 8px;
border: 1px solid #ccc;
border-radius: 4px;
box-sizing: border-box;
font-size: 16px;
margin-bottom: 10px;
}

button[type="submit"] {
background-color: #4CAF50;
color: white;
padding: 12px 20px;
border: none;
border-radius: 4px;
cursor: pointer;
font-size: 16px;
}

button[type="submit"]:hover {
background-color: #45a049;
}

.mainAttendanceModal {
/* position: fixed;
top: 0;
left: 0; */
width: 100%;
height: 100%;
display: flex;
flex-direction: column;
justify-content: center;
align-items: center;
}

.mainAttendanceModal table {
width: 90%;
margin: auto;
border: none;
background-color: #ccc;
}

.mainAttendanceModal th {

```

```

        background-color: #4CAF50;
        color: white;
        border-collapse: collapse;
        padding: 10px;
    }

    .mainAttendanceModal td {
        padding: 10px;
        border-collapse: collapse;
        /* text-align: center; */
        border: none;
        /* font-weight: bolder; */
    }
</style>
</head>

<body>
    <% if(title !=="form" ) { %>
        <style>
            body {
                background-color: rgba(0, 0, 0, 0.7);
            }
        </style>
        <% } %>

        <% if(title==="form" ) { %>
            <form action="/dailyreport" method="post">
                <label for="date">Choose one date:</label>
                <input type="date" name="date" id="date">
                <button type="submit" class="">Submit</button>
            </form>
        <% } else { %>
            <form action="/dailyreport" class="tableDate" method="post">
                <input type="date" name="date" id="date" value="<%= date %>">
                <button type="submit" class="">Submit</button>
            </form>
            <br>
            <div class="mainAttendanceModal">
                <table id="studentRecords">
                    <tr>
                        <th>Roll No.</th>
                        <th>Firstname</th>
                        <th>Lastname</th>
                        <th>Department</th>
                        <th>Subject Code</th>
                        <th>Entry at</th>
                    </tr>
                    <% if(attendance==="404 not found" ) { %>
                        <h1 style="color: white;">No Record Found for Date : <%= date %>
                    </h1>
                    <% } else { %>
                        <% attendance.forEach((item)=> { %>
                            <tr>
                                <td>
                                    <%= item.roll %>
                                </td>
                                <td>
                                    <%= item.fname %>
                                </td>
                            </tr>
                        <% } %>
                    <% } %>
                </table>
            </div>
        <% } %>
    </body>
</html>

```



```

        </td>
        <td>
            <%= item.lname %>
        </td>
        <td>
            <%= item.department %>
        </td>
        <td>
            <%= item.scode %>
        </td>
        <td>
            <%= item.time %>
        </td>
    </tr>
    <% }) %>
    <% } %>
</table>
</div>
<% } %>
</body>

</html>

```

studentRegister.ejs

```

<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Student Registration Form</title>

    <style>

        * {

            box-sizing: border-box;

            margin: 0;

            padding: 0;

        }

        body {

            font-family: Arial, sans-serif;

            background-color: #212121;

            color: #fff;

        }

        .form-container {

            display: flex;

            justify-content: center;

```

```
    align-items: center;
    min-height: 100vh;
}
.form-wrapper {
    width: 100%;
    max-width: 500px;
    padding: 40px;
    border: 1px solid #91f4fa;
    border-radius: 10px;
    background-color: #424242;
}
h1 {
    font-size: 32px;
    font-weight: bold;
    margin-bottom: 20px;
}
label {
    font-size: 18px;
    font-weight: bold;
    margin-bottom: 10px;
}
input[type=text] {
    display: block;
    width: 100%;
    padding: 10px;
    margin-bottom: 20px;
    border: none;
    outline: 1px solid #91f4fa;;
    border-radius: 5px;
    background-color: #616161;
    color: #fff;
}
input[type=submit] {
    display: block;
```

```

width: 100%;
padding: 10px;
border: none;
border-radius: 5px;
background-color: #00c8ff;
color: #000000;
font-size: 18px;
font-weight: bold;
cursor: pointer;
}
</style>
</head>
<body>
<div class="form-container">
<div class="form-wrapper">
<h1>Student Registration Form</h1>
<form action="/register" method="POST">
<label for="roll">Roll No.:</label>
<input type="text" id="roll" name="roll">
<label for="fname">First Name:</label>
<input type="text" id="fname" name="fname">
<label for="lname">Last Name:</label>
<input type="text" id="lname" name="lname">
<label for="department">Department:</label>
<input type="text" id="department" name="department">
<label for="scode">Subject Code:</label>
<input type="text" id="scode" name="scode">
<input type="submit" value="Submit">
</form>
</div>
</div>
</body>
</html>

```

FACT SHEET OF COMPONENTS

- ❖ **NodeMCU** : The ESP8266 NodeMCU CP2102 board is a popular development board based on the ESP8266 Wi-Fi module. It is widely used in IoT (Internet of Things) projects and prototyping due to its low cost, ease of use, and built-in Wi-Fi capabilities. While I can't provide a step-by-step guide on how to manufacture the board, I can give you an overview of the components and processes involved in its manufacturing.

1. Components:

- ESP8266 Wi-Fi module: The heart of the NodeMCU board, responsible for the Wi-Fi connectivity and processing capabilities.
- CP2102 USB-to-Serial chip: Enables communication between the board and a computer via USB.
- Voltage regulator: Provides stable power supply to the board.
- Crystal oscillator: Generates clock signals for precise timing.
- Capacitors and resistors: Used for filtering, decoupling, and voltage regulation.
- LEDs and buttons: Provide visual feedback and user input options.
- PCB (Printed Circuit Board): The board itself, with copper traces connecting the components.

2.Details :

- Serial/USB Chip: CP2102
- Output Power(dBm) : 19.5 @802.11b Mode
- Flash Memory(Mb): 4
- Support: SDIO 1.1/2.0, SPI, UART
- Length (mm): 49
- Width (mm) : 24
- Height (mm): 13
- Weight (gm): 8
- Shipment Weight: 0.012 kg
- Shipment Dimensions: 6 × 3.5 × 2.5 cm
- Operating temperature : around 25 [normal temperation]
- Price : 230

3. Manufacturing processes:

- PCB design: The board's circuitry and layout are designed using PCB design software. This includes placing the components, routing

the copper traces, and defining the board's dimensions.

- **PCB fabrication:** The designed PCB layout is sent to a manufacturer who produces the physical PCBs using specialized processes, such as etching the copper layers, drilling holes, and applying solder mask and silkscreen layers.
- **Component placement:** Once the PCBs are ready, the components are placed on the board using automated pick-and-place machines. These machines accurately position the components on their designated pads.
- **Soldering:** The board undergoes a reflow soldering process, where solder paste is applied to the pads, and the board is heated to melt the solder, creating a strong electrical connection between the components and the PCB.
- **Testing:** After soldering, the boards are tested to ensure proper functionality. This may involve flashing firmware onto the ESP8266 module, checking USB communication, and verifying Wi-Fi connectivity.
- **Assembly:** Once the boards pass the testing phase, they are typically assembled into their final form, which may include adding headers, connectors, and other accessories as required.
- **Packaging:** The manufactured boards are packaged and prepared for distribution, which may involve placing them in anti-static bags, attaching user manuals, and adding labels.

It's important to note that the above process is a general overview, and specific manufacturers may have variations in their manufacturing processes. The NodeMCU board is widely available from various suppliers, so if you're interested in obtaining one, you can purchase it from electronic component distributors or online marketplaces.

- ❖ **RC522 RFID Card Reader:** The RC522 RFID Card Reader Module is a popular RFID (Radio Frequency Identification) module that operates at a frequency of 13.56MHz. It is commonly used in various applications such as access control systems, attendance systems, and inventory management. Here's an overview of the components and processes involved in the manufacturing of an RC522 RFID Card Reader Module:

1. Components:

- - RC522 RFID chip: The main integrated circuit that handles the RFID communication and protocols.
- - Antenna: The coil-shaped component that emits and receives the RF signals for reading and writing data to RFID cards.
- - Voltage regulator: Provides stable power supply to the module.
- - Capacitors and resistors: Used for filtering, decoupling, and voltage regulation.
- - Inductors: Help in tuning the RF circuit and impedance matching.
- - PCB (Printed Circuit Board): The board that holds and interconnects the components.

2. Details:

- Model: MF522-ED
- Working current @3.3V : 13 – 26mA
- Standby current @3.3V : 10 – 13mA
- Sleep current: $<80\frac{1}{4}A$
- Peak current : $<30mA$
- Working frequency : 13.56MHz
- Card reading distance : 0-60mm
- Protocol : SPI
- Data communication speed : 10 Mbit/s Max
- Operating Temperature (°C) : -20 to 80
- PCB Size (L x W) mm : 60 x 40
- Shipment Weight : 0.07 kg
- Shipment Dimensions : $8 \times 5 \times 2$ cm
- Price : 100

3. Manufacturing processes:

- - PCB design: The circuitry and layout of the module are designed using PCB design software. This involves placing the components, routing the traces, and defining the dimensions of the board.
- - PCB fabrication: The designed PCB layout is sent to a manufacturer who produces the physical PCBs using processes such as copper etching, drilling holes, and applying solder mask and silkscreen layers.
- - Component placement: Once the PCBs are ready, the components are placed on the board using automated pick-and-place machines. These machines accurately position the components on their designated pads.
- - Soldering: The board undergoes a reflow soldering process where

solder paste is applied to the pads, and the board is heated to melt the solder, creating electrical connections between the components and the PCB.

- - Testing: After soldering, the modules go through functional testing to ensure proper operation. This may involve checking the communication with RFID cards, verifying voltage regulation, and validating the module's output signals.
- - Assembly: Once the modules pass the testing phase, they are typically assembled into their final form, which may include adding headers, connectors, and mounting options.
- - Packaging: The manufactured modules are packaged and prepared for distribution, which may involve placing them in anti-static bags, attaching user manuals, and adding labels.

It's important to note that the manufacturing process can vary between different suppliers and manufacturers. If you're interested in obtaining an RC522 RFID Card Reader Module, you can purchase it from electronic component distributors or online marketplaces.

- ❖ **LED** : The manufacturing process of a 5mm DIP (Dual Inline Package) LED involves several steps. Here's a general overview of the process:

1. Semiconductor Wafer Preparation:

- - Creation of the semiconductor material: The LED manufacturing process typically starts with the creation of a wafer made of a suitable semiconductor material, such as gallium arsenide (GaAs) or gallium phosphide (GaP).
- - Crystal growth: The wafer undergoes a crystal growth process, where the semiconductor material is grown in a controlled environment to achieve the desired crystal structure and composition.

2. Epitaxy:

- Epitaxial growth: The wafer is subjected to epitaxial growth, a process where additional layers are deposited on the wafer's surface. These layers are typically composed of different semiconductor materials, such as n-doped and p-doped layers, to create the necessary junctions for LED operation.

3. Lithography and Etching:

- - Photolithography: A photosensitive material called a photoresist is applied to the wafer's surface. A photomask, which defines the pattern for LED formation, is used to expose the photoresist to UV light.
- - Etching: Chemical or plasma etching is performed to remove the exposed or unexposed portions of the photoresist, leaving behind the desired pattern on the wafer.

4. Contact Formation:

- - Metal deposition: Metal layers, such as aluminum or gold, are deposited onto the wafer's surface to create electrical contacts for the LED.
- - Patterning and etching: Photolithography and etching processes are used to define the contact areas and remove excess metal, leaving only the desired contact patterns.

5. Dicing:

- Wafer dicing: The wafer is diced into individual LED chips using a diamond saw or laser cutting. Each chip represents an individual LED.

6. Packaging:

- - Die attachment: Each LED chip is attached to a lead frame or substrate using a die bonding process.

❖ **Buzzer:** A buzzer is an electronic device that produces sound when powered, commonly used in alarms, notifications, and feedback systems. It operates by vibrating a piezoelectric element or electromagnetic coil to generate audible tones. Here's a general overview of the process:

1. Components

- Piezoelectric Disc/Electromagnetic Coil: Core for sound production.
- Casing: Protects and amplifies sound.
- Driver Circuit (Active Only): Generates a fixed tone.

- Pins/Wires: Connects to the circuit.

2. Specifications

- Operates at 3V–12V DC with a frequency range of 1–5 kHz and sound pressure of 80–100 dB.
- Lightweight (2–10 g), compact (10–30 mm diameter), and operates within -20°C to 60°C.

3. Applications

Used in alarms, electronic devices, toys, and industrial systems for audio alerts.

4. Manufacturing Steps

Material selection, component fabrication, assembly, sound testing, and packaging.

5. Advantages

Simple, reliable, low power, cost-effective, and compact.

6. Limitations

Fixed tones (active type), limited durability, and restricted sound range.

The buzzer is a simple yet versatile component widely used in various applications for sound alerts and notifications. Manufacturing processes may vary among suppliers, and buzzers are readily available through electronic component distributors or online marketplaces.